the data into a useful hash that the React page can consume.

The actual view page then pretty much vanishes:

appendix_react/01/app/views/schedules/show.html.erb

```
<div
  id="schedule-react-element"
  data-favorite-channel-name="<%= Turbo::StreamsChannel.signed_stream_name(
                                  [current_user, :favorites]
                                ) %>">
</div>
```

Similar to the other React app, we're creating a `div` element with a known ID that we'll use to attach the React app. We've put one value in the data list for the element—the encrypted name of the ActionCable channel that will manage the user favorites. More on that in a bit.

To invoke the element, we add another section to the event handler that fires when the page loads:

appendix_react/01/app/javascript/components/venue_display.tsx

```
const schedule_element = document.getElementById("schedule-react-element")
if (schedule_element) {
  const root = createRoot(schedule_element)
  root.render(
    <ScheduleApp
      favoriteChannelName={schedule_element.dataset.favoriteChannelName}
    />
  )
}
```

It looks for the correct DOM element, and if that element exists, attaches the React app to the element.

You can assume, by the way, that all the elements need to be included into their various files, and that all types need to be declared (I won't necessarily be showing all the boilerplate).